

N 皇后问题求解实验报告

摘要

本实验报告旨在探究经典的 N 皇后问题的求解方法，并基于提供的代码实现一个具备回溯法自动求解和可视化手动放置功能的交互式应用。实验核心是理解和实现回溯算法，同时利用 tkinter 和 matplotlib 库构建用户界面，实现解的可视化展示以及冲突检测的交互式棋盘。结果显示，该方法能有效地找到 N 皇后问题的所有可行解，并提供了直观的交互体验。

1. N 皇后问题的求解核心方法

N 皇后问题要求在 $N \times N$ 的棋盘上放置 N 个皇后，使得任何两个皇后都不能互相攻击，即任意两个皇后不能位于同一行、同一列或同一对角线上。

1.1 核心算法：回溯法

本代码采用的核心算法是回溯法，它是一种深度优先搜索的特殊应用，用于系统地搜索解空间树中的所有可能解。

1.1.1 基本思想

逐行放置：算法从第一行开始，尝试在当前行放置一个皇后。

安全性检查：对于当前行 `row` 的每一列 `col`，检查该位置是否安全即不被已放置的皇后攻击。

递归与剪枝：如果位置安全，则将皇后放置在此处，并递归地进入下一行继续尝试。这一步相当于在解空间树中向下延伸。如果当前行尝试完所有列都没有找到安全位置，或下一行递归返回后，程序需要回溯：撤销当前行的皇后放置，然后尝试当前行的下一列。

终止条件：当所有 N 个皇后都成功放置，则找到一个完整解并记录。

1.2 关键冲突检测

为了高效地判断一个位置是否安全，我们不需要逐个检查已放置的皇后，而是利用数组来记录行、列和对角线的占用情况。

列占用：记录哪一列已经被占用。

主对角线占用：主对角线上的格子满足 $row - col = \text{常数}$ 。为了方便用数组索引，我们使用 $d1 = row + col$ 作为索引。N * N 棋盘共有 $2N-1$ 条主对角线，索引范围是 $[0, 2N-2]$ 。

副对角线占用：副对角线上的格子满足 $row + col = \text{常数}$ 。我们使

用 $d2 = \text{row} - \text{col} + N - 1$ 作为索引。 $N \times N$ 棋盘共有 $2N-1$ 条副对角线，索引范围是 $[0, 2N-2]$ 。

1.2.1 关键代码

以下是实现回溯法和冲突检测的核心函数及其内部的递归函数

```
def solve_n_queens(n):

    def search(row, col_used, diag1_used, diag2_used, solution, solutions):

        if row == n:

            solutions.append(solution.copy())

            return

        for col in range(n):

            d1 = row + col

            d2 = row - col + n - 1

            if not col_used[col] and not diag1_used[d1] and not diag2_used[d2]:

                solution[row] = col

                col_used[col] = diag1_used[d1] = diag2_used[d2] = True

                search(row + 1, col_used, diag1_used, diag2_used, solution, solutions)

                col_used[col] = diag1_used[d1] = diag2_used[d2] = False

        solutions = []

        search(0, [False]*n, [False]*(2*n-1), [False]*(2*n-1), [0]*n, solutions)

    return solutions
```

2. 可视化与用户界面实现

本应用利用 tkinter 构建图形用户界面，并使用 matplotlib 进行棋盘的可视化绘制。

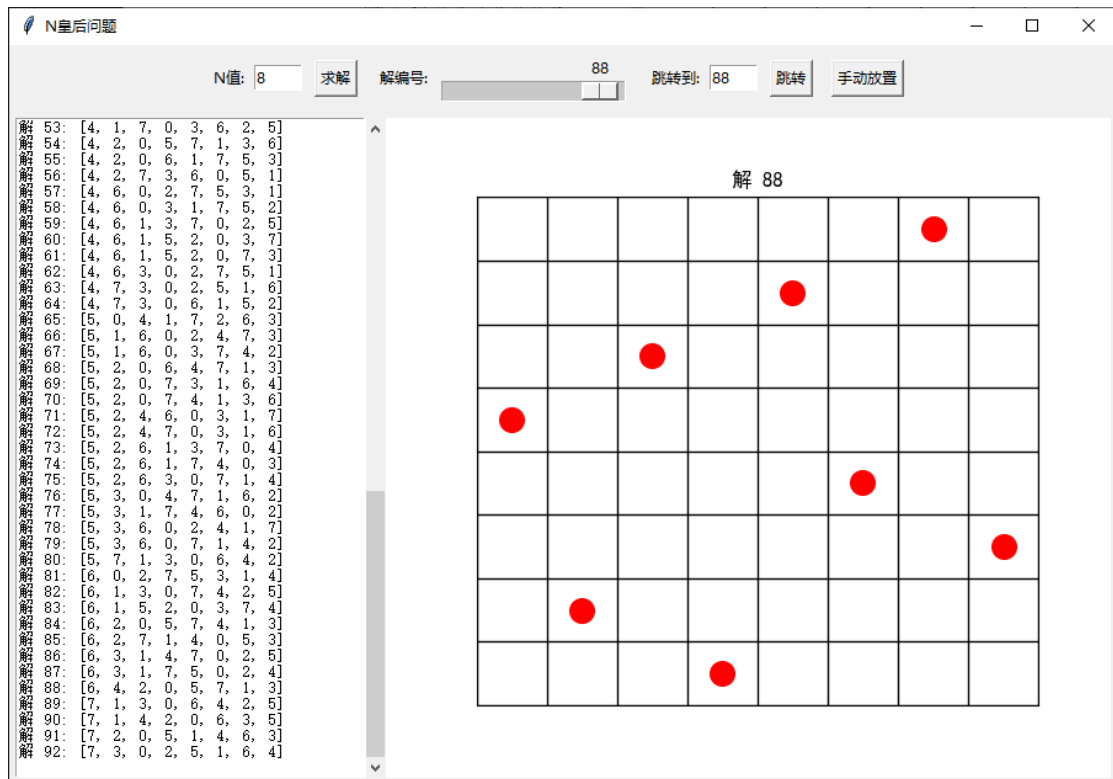
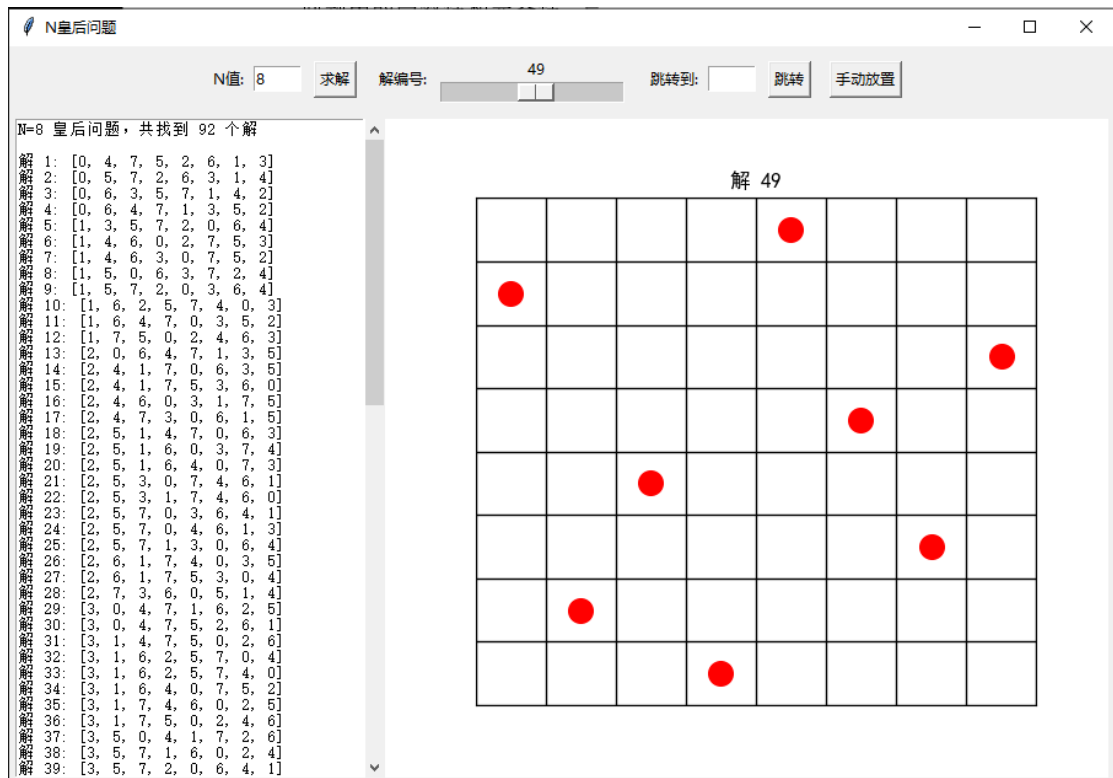
2.1 求解结果可视化

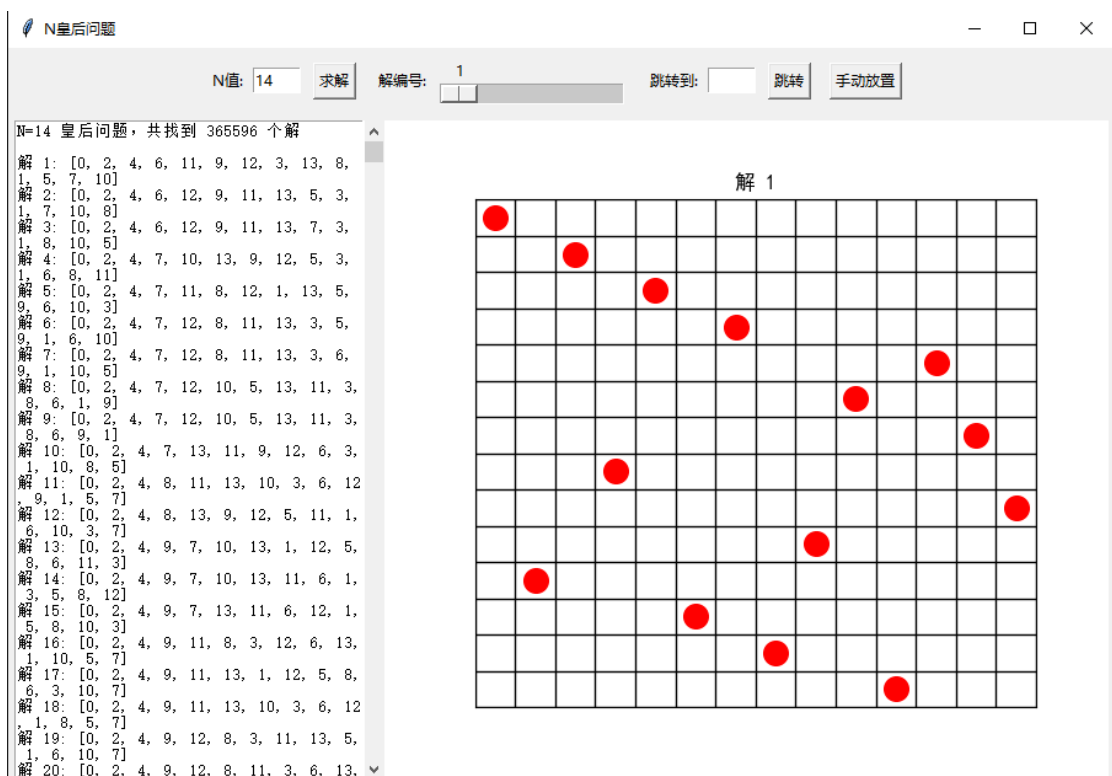
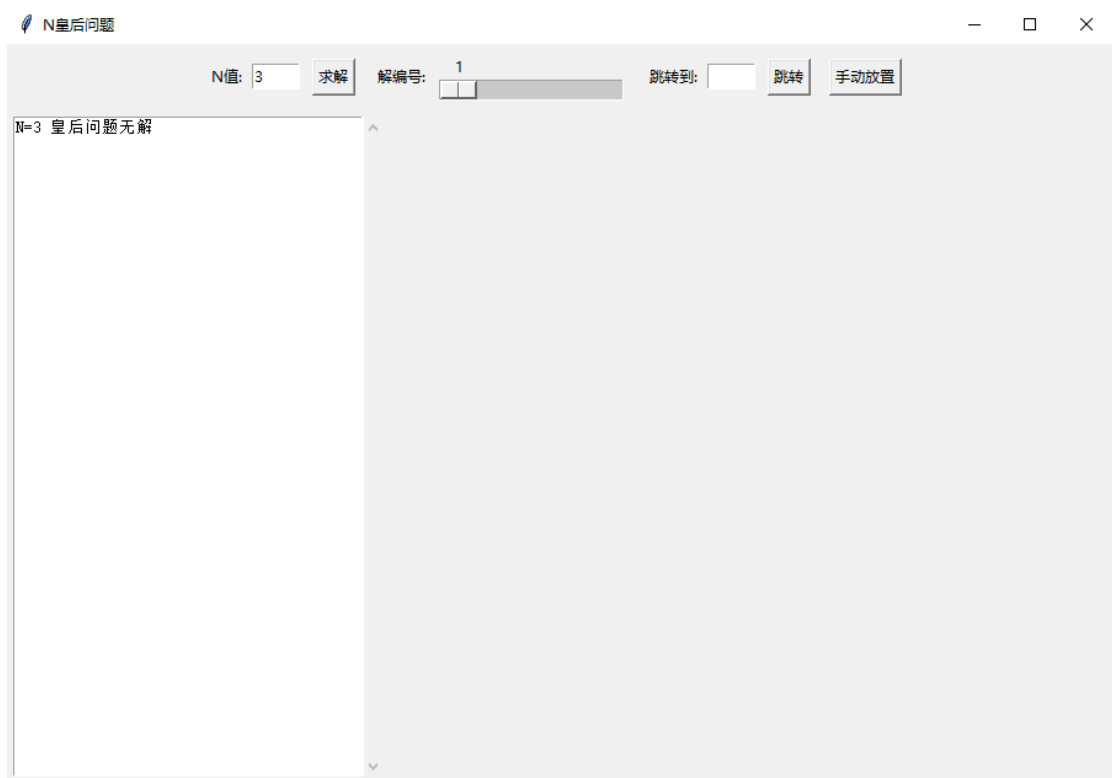
函数 draw_solution 负责将找到的一个完整解 solution 绘制成 $N \times N$ 棋盘图。

棋盘绘制： 使用 ax.axvline 和 ax.axhline 绘制棋盘格线。

皇后标记： 使用 ax.scatter 和 ax.text 在指定的位置放置一个红色的圆点并标上字母 "Q"，表示皇后。注意坐标系转换： row 索引从 0 开始，从上往下

数，而绘图的 y 轴通常从下往上递增，所以需要用到 $N - 0.5 - \text{row}$ 来映射 y 坐标。





2.2 交互式手动放置可视化

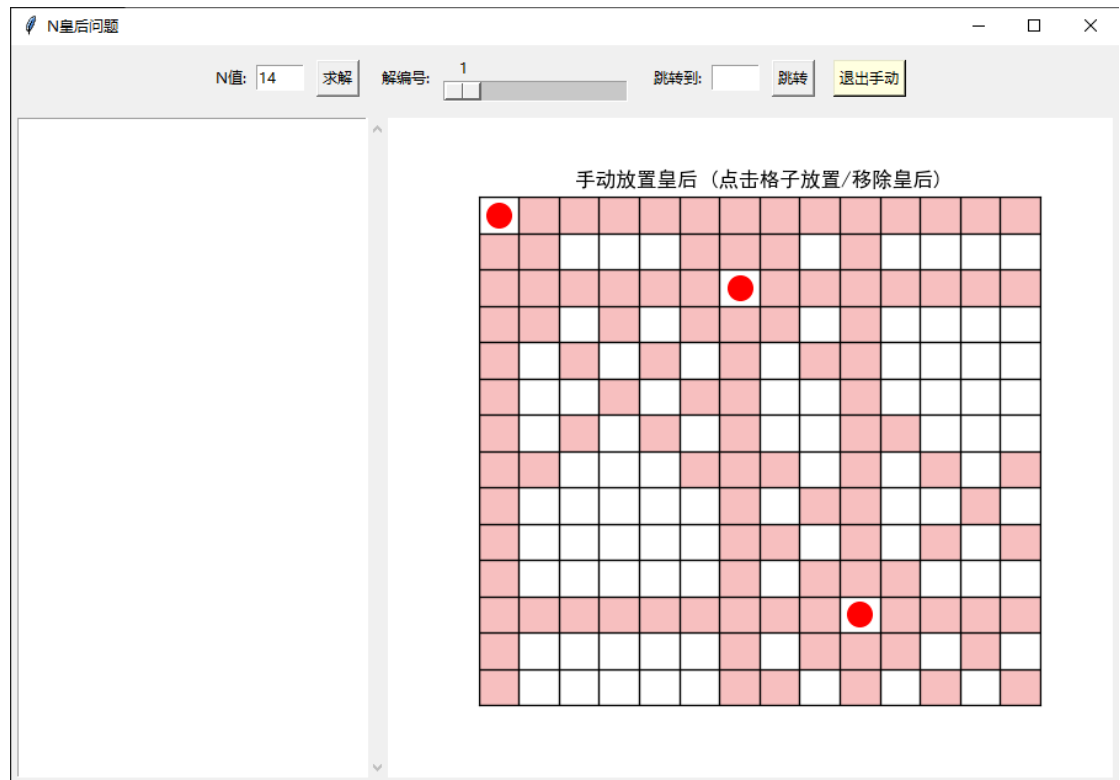
函数 `draw_manual_board` 和类方法 `on_click` 共同实现了交互式棋盘功能：

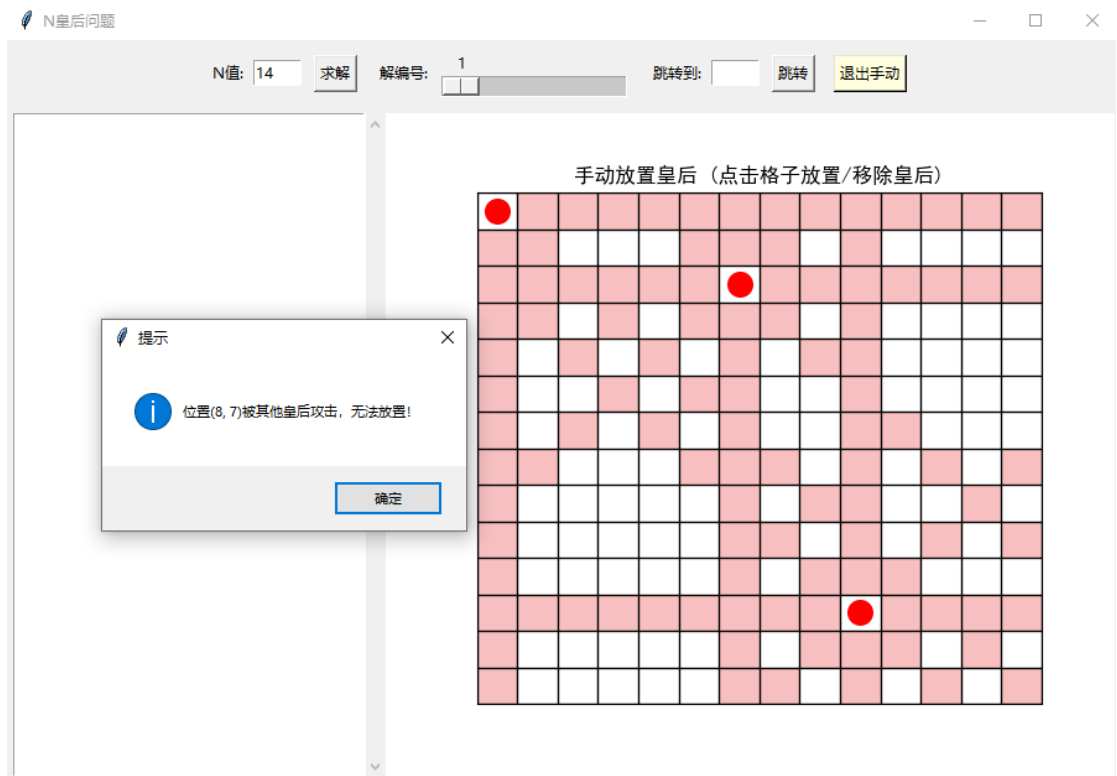
绘制冲突：`calculate_conflicts` 方法计算出当前棋盘上所有已被放置的皇后攻击的格子，并在 `draw_manual_board` 中使用浅红色 矩形标记出来，为用

户提供直观的反馈。

点击交互： `on_click` 方法捕获用户点击事件，将点击的屏幕坐标 转换为棋盘的坐标。

安全放置原则： 在手动模式下，只有当用户点击一个非冲突位置时，才能放置或移除皇后。如果点击了冲突位置，程序会弹出提示框，体现了对 N 皇后规则的实时校验。





2.2.1 关键代码

```
def draw_solution(solution, n, title=""):

    fig, ax = plt.subplots(figsize=(6, 6))

    ax.set_title(title, fontsize=12)

    for i in range(n + 1):

        ax.axvline(i, color='black', lw=1)

        ax.axhline(i, color='black', lw=1)

    ax.set_xlim(0, n)

    ax.set_ylim(0, n)

    ax.set_xticks([])

    ax.set_yticks([])

    for row, col in enumerate(solution):

        ax.scatter(col + 0.5, n - 0.5 - row, s=200, color='red', zorder=5)

        ax.text(col + 0.5, n - 0.5 - row, "Q", color='white',

                ha='center', va='center', fontweight='bold')
```

```

    return fig

def calculate_conflicts(self):
    conflicts = set()

    for row, col in self.queens:
        for c in range(self.n):
            if c != col:
                conflicts.add((row, c))

        for r in range(self.n):
            if r != row:
                conflicts.add((r, col))

        r, c = row-1, col-1
        while r >= 0 and c >= 0:
            conflicts.add((r, c))
            r -= 1
            c -= 1

        r, c = row+1, col+1
        while r < self.n and c < self.n:
            conflicts.add((r, c))
            r += 1
            c += 1

        r, c = row-1, col+1
        while r >= 0 and c < self.n:
            conflicts.add((r, c))
            r -= 1
            c += 1

        r, c = row+1, col-1
        while r < self.n and c >= 0:

```

```
conflicts.add((r, c))

r += 1

c -= 1

return conflicts
```

3. 总结、分析与实验心得

3.1 总结与分析

回溯法穷举了所有可能的皇后放置方案，并通过冲突检测避免了无效的搜索路径，保证了能找到所有可行解，体现了回溯算法在约束满足问题中的高效性和完备性。

N 皇后问题的解空间大小是 N^N ，但由于回溯法的剪枝操作，实际的搜索次数远小于 $N!$ 。代码通过使用 $O(1)$ 时间复杂度的数组查找来判断冲突，极大地提升了搜索效率。

结合 tkinter 和 matplotlib 实现了友好的 GUI。特别是手动放置模式，通过实时检测并标记冲突区域，能够直观地理解 N 皇后问题的约束条件，具有优秀的教学和演示价值。

3.2 实验心得

N 皇后问题中最巧妙且关键的部分在于对角线的索引计算。通过 $d_1 = \text{row} + \text{col}$ 和 $d_2 = \text{row} - \text{col} + N - 1$ 的转换，将二维的对角线冲突判断，简化为一维数组的占用布尔值检查，深刻体会到将几何约束转化为代数表示的强大。

递归调用前的标记和递归返回后的撤销标记是回溯法的精髓。它确保了状态的隔离和重置，使得每次尝试都是在“干净”的环境下进行，是实现完整搜索的关键。

结合 tkinter 和 matplotlib 需要处理好图形嵌入 和事件处理机制，尤其是在坐标转换方面，将 Matplotlib 的 y 轴递增坐标系映射到棋盘的 0 行在顶部的约定，需要细心处理。